Random Forest Classification of Network Traffic Attack Categories

Student number: 26905167
Stellenbosch University

***Abstract*—The classification of network traffic records into attack categories is a core task of intrusion detection. The study evaluated a random forest on a deduplicated version of the UNSW-NB15 dataset, and compared the forest with a bagged ensemble of trees, a classification tree and a k-nearest neighbours model. The forest was selected because the forest shares the base learner of the tree, so that the comparison isolates the effect of ensembling. Five research questions guided the tuning of the forest. All pre-processing was fitted within cross-validation folds on a development partition, and every model was evaluated once on a held-out generalisation set. The forest attained a macro-averaged F_1 measure of 0.607, against 0.591 for the tree, 0.589 for bagging and 0.459 for the nearest neighbour model, and the improvement over the tree was statistically significant. The random feature subset produced the entire improvement, because bagging alone did not outperform the tree. The analysis further established that $7.1\,\%$ of the records share identical features with a record of a different category. The conflicts render two attack categories inseparable and mask most of the advantage of the forest on the separable records.**

I. Introduction

Network intrusion detection requires the assignment of individual traffic records to attack categories. Manual inspection does not scale to the volume of traffic carried by a modern network, so an intrusion detection system (IDS) relies on automated classification of traffic records. Supervised learning provides the mechanism, in that a classifier learns the association between measured flow characteristics and known attack categories.

A preceding study compared two single classifiers on an edited version of the UNSW-NB15 dataset, described by Moustafa and Slay [20]. A classification tree outperformed k-nearest neighbours (KNN) classification on every performance measure that accounts for class imbalance. The tree nevertheless detected the backdoor and analysis categories in fewer than half of the cases, and the variation of a single tree under perturbation of the training data remained unexamined. Single classification trees are known to be unstable, in the sense that small changes to the training set produce large changes to the induced tree [2].

The present study investigates ensemble learning as a remedy. A random forest (RF), introduced by Breiman [3], was selected as the ensemble method, and was compared against revised implementations of the classification tree and the KNN model. The RF shares the base learner of the classification tree. Any difference in performance between the two models is therefore attributable to the ensemble mechanism, rather than to a change of hypothesis space. A bagged ensemble of trees was evaluated as an intermediate model, so that the contribution of bootstrap aggregation was separated from the contribution of random feature selection.

The tuning of the RF was designed to answer five research questions, fixed before any forest was trained:

1) Does the RF improve on the tuned classification tree, and on which attack categories does the improvement occur?
2) How does performance change with the number of trees, and does the ensemble overfit as trees are added?
3) How does the size of the random feature subset trade the strength of individual trees against the correlation between trees?
4) Do the depth and leaf size limits required by the single tree remain beneficial inside an ensemble?
5) Does instance weighting in inverse proportion to class frequency remain beneficial once predictions are averaged over many trees?

The preceding study fitted selected pre-processing steps on the full dataset and retained no held-out data. The present study removes both limitations: every fitted transformation was learned inside the cross-validation folds, and every model was evaluated once on a generalisation set that played no part in any decision.

The remainder of the report is organised as follows. Section II presents the background on the three classification approaches, on ensemble learning, and on the evaluation of classifiers. Section III describes the dataset and the pre-processing. Section IV describes the implementation. Section V describes the empirical process. Section VI presents and discusses the results, and Section VII concludes the report.

II. Background

The section provides the conceptual background required for the remainder of the report. The section first describes the two single classifiers of the preceding study, then presents ensemble learning in general and the RF in particular, and finally describes the measures and statistical tests used to compare classifiers.

A. Nearest neighbour classification

The KNN algorithm, analysed by Cover and Hart [5], is an instance-based classifier. Training consists of the storage of the training set. Classification of an unseen instance proceeds by the retrieval of the k stored instances closest to the unseen instance, followed by a vote among the retrieved instances.

For two instances represented by the feature vectors $\mathbf{x}$ and $\mathbf{z}$ over p features, the Euclidean distance is

$$\delta(\mathbf{x},\mathbf{z}) = \sqrt{\sum_{j=1}^{p}(x_j - z_j)^2}, \tag{1}$$

where x_j denotes the value of feature j of instance $\mathbf{x}$. Equation (1) sums squared differences of raw feature values, so a feature of large numeric range dominates the distance irrespective of the usefulness of the feature for discrimination. Feature scaling is therefore a prerequisite of the algorithm.

Two control parameters govern the behaviour of the algorithm. A small value of k produces a flexible decision boundary that is sensitive to noise, whereas a large value of k smooths the boundary and biases prediction toward the majority class. Under uniform weighting each retrieved neighbour casts one vote. Under distance weighting each vote is scaled by the reciprocal of δ, so that closer neighbours exert greater influence. Aggarwal *et al.* [1] showed that the contrast between the nearest and the furthest neighbour diminishes as the number of dimensions grows, which penalises representations with many sparse features.

B. Classification trees

A classification tree, described by Breiman *et al.* [4], recursively partitions the feature space. Each internal node tests one feature against a threshold, and each leaf assigns a class label. Construction is greedy: at each node the algorithm selects the feature and threshold that maximise the reduction of an impurity measure. For a node at which π_j denotes the proportion of instances of class j, and c denotes the number of classes, the Gini index and the entropy are

$$G = 1 - \sum_{j=1}^{c} \pi_j^2 \quad \text{and} \quad H = -\sum_{j=1}^{c} \pi_j \log_2 \pi_j. \tag{2}$$

Entropy was employed by Quinlan [25] as the split criterion of an early tree induction algorithm.

A tree grown until every leaf is pure fits the training set exactly and generalises poorly. Pre-pruning limits growth through a maximum depth and a minimum number of instances per leaf. Post-pruning grows a full tree and then removes subtrees. Cost-complexity pruning [4] selects the subtree of the full tree h that minimises

$$R_\alpha(h) = R(h) + \alpha |h|, \tag{3}$$

where $R(h)$ denotes the weighted training misclassification cost of h, $|h|$ denotes the number of leaves of h, and $\alpha \geq 0$ denotes the complexity penalty. Every removed subtree is replaced by a single leaf that predicts the majority class of the training instances that reach the removed subtree. Increasing values of α therefore produce progressively smaller trees.

Class imbalance is addressed through instance weights. Under weighting in inverse proportion to class frequency, an instance of class j receives the weight $w_j = n/(c\,n_j)$, where n denotes the number of training instances and n_j denotes the number of training instances of class j. The weights enter the class proportions of Equation (2), so that a split which isolates a rare class achieves a larger impurity reduction [14].

C. Ensemble learning

An ensemble combines the predictions of several base classifiers into a single prediction. Dietterich [7] identified three reasons for the success of ensembles. The statistical reason is that a finite training set supports many hypotheses of equal training accuracy, and an average over the hypotheses reduces the risk of the selection of a poor hypothesis. The computational reason is that greedy induction reaches a local optimum, and different starting conditions reach different optima. The representational reason is that a combination of hypotheses represents decision boundaries that no single member represents.

The error of a learner decomposes into a bias term and a variance term [13]. Fully grown trees have low bias and high variance, which makes the trees ideal members of an averaging ensemble: the average retains the low bias and reduces the variance. An average reduces variance only to the extent that the members disagree. Kuncheva and Whitaker [18] reviewed measures of disagreement between members, termed diversity, and examined the relationship between diversity and ensemble accuracy.

Three families of ensemble methods dominate the literature. Bagging and random forests train members independently on perturbed versions of the training set and average the predictions. Boosting, of which AdaBoost by Freund and Schapire [11] and gradient boosting by Friedman [12] are the best known instances, trains members sequentially, so that each member concentrates on the errors of the preceding members. Stacking, introduced by Wolpert [28], trains a second-level learner on the outputs of heterogeneous members.

D. Bagging

Bagging, introduced by Breiman [2], trains τ members on τ bootstrap samples of the training set. A bootstrap sample is drawn with replacement and has the size of the training set, so each sample omits approximately $36.8\,\%$ of the distinct training instances. The members vote, or average the class probabilities, to produce the prediction of the ensemble. Breiman [2] showed that bagging improves unstable learners, such as classification trees, and leaves stable learners, such as the KNN algorithm, largely unchanged.

E. Random forests

The RF [3] extends bagging with a second source of randomisation. At every node of every tree, the best split is sought among a random subset of q of the p features, rather than among all features. The idea follows the random subspace method of Ho [15]. A value of q equal to p reduces the RF to bagging.

Breiman [3] bounded the generalisation error ε of a forest by

$$\varepsilon \leq \frac{\bar{\rho}\,(1 - s^2)}{s^2}, \tag{4}$$

where s denotes the strength of the individual trees, defined as the expected margin of a tree, and $\bar{\rho}$ denotes the mean correlation between trees. Equation (4) identifies q as the central control parameter of the method. A small value of q decorrelates the trees, which reduces $\bar{\rho}$, but also weakens each tree, which reduces s. The best value of q balances the two effects and depends on the dataset [24].

Breiman [3] additionally established, through the strong law of large numbers, that the generalisation error converges as τ grows. An increase of τ therefore cannot cause overfitting, and the choice of τ trades computational cost against the residual variance of the ensemble. Probst and Boulesteix [23] showed that the performance curve over τ is not always monotone for measures other than the error rate, which motivates an empirical examination of the curve rather than an assumption.

The trees of an RF are conventionally grown to purity, because the average removes the variance that pruning would otherwise control. Probst *et al.* [24] reported that a limit on the leaf size occasionally improves performance, particularly on noisy data. Class weighting applies to the forest as to a single tree, either with weights computed once from the full training set or with weights recomputed for every bootstrap sample.

The RF also offers a measure of feature importance, computed as the total weighted impurity reduction attributable to each feature. Strobl *et al.* [27] showed that the impurity-based measure favours features with many candidate split points. Permutation importance, defined as the reduction of a performance measure after the random permutation of the values of one feature in a held-out set, avoids the bias.

F. Performance measures

Let $\mathbf{C}$ denote the confusion matrix of a classifier on a test set, where the element C_{jl} counts the instances of class j predicted as class l. The precision a_j, the recall r_j and the F_1 measure F_j of class j are

$$a_j = \frac{C_{jj}}{\sum_{l=1}^{c} C_{lj}}, \quad r_j = \frac{C_{jj}}{\sum_{l=1}^{c} C_{jl}}, \quad F_j = \frac{2\,a_j r_j}{a_j + r_j}. \tag{5}$$

The macro-averaged F_1 measure $\bar{F}$, the balanced accuracy $\bar{r}$ and the accuracy η are

$$\bar{F} = \frac{1}{c}\sum_{j=1}^{c} F_j, \quad \bar{r} = \frac{1}{c}\sum_{j=1}^{c} r_j, \quad \eta = \frac{\sum_{j=1}^{c} C_{jj}}{\sum_{j=1}^{c}\sum_{l=1}^{c} C_{jl}}. \tag{6}$$

The accuracy η weights every instance equally, and is therefore dominated by the largest classes. The measures $\bar{F}$ and $\bar{r}$ weight every class equally, which suits problems in which the detection of rare classes is the objective [26].

G. Statistical comparison of classifiers

Dietterich [6] recommended the McNemar test [19] for the comparison of two classifiers evaluated on a single test set. Let u denote the number of test instances classified correctly by the first classifier only, and v the number classified correctly by the second classifier only. Under the null hypothesis of equal error rates, the statistic

$$\chi^2 = \frac{(|u-v|-1)^2}{u+v} \tag{7}$$

follows a chi-squared distribution with one degree of freedom. The McNemar test addresses accuracy only. A difference in $\bar{F}$ is instead assessed with the paired bootstrap [9]: test instances are resampled with replacement, the difference in $\bar{F}$ between the two classifiers is recomputed on every resample, and the percentiles of the resulting distribution yield a confidence interval. When several pairwise tests are performed, the procedure of Holm [16] adjusts the p-values so that the probability of one or more false rejections across the family remains below the significance level.

III. Data and pre-processing

The section describes the dataset, the properties of the dataset that bear on the choice and configuration of the models, and the pre-processing applied to the input of each model. The section first reports the data quality analysis, then describes the corrections shared by all models, and finally describes the model-specific pre-processing candidates that were evaluated.

A. Dataset

The supplied file contains 257,673 network flow records, each described by a record identifier, 42 descriptive features and a target feature. The target feature assigns every record to one of ten categories, namely normal traffic and nine attack categories. The descriptive features comprise three nominal features, namely the transaction protocol, the connection state and the application service, and 39 numeric features that describe packet counts, byte counts, timing, and connection statistics [21].

The data quality analysis of the preceding study established eight defects. The record identifier is unique and ordered by category. The service feature contains 141,321 missing values. The binary feature that indicates a file transfer protocol (FTP) login contains 26 values greater than one. Numeric ranges span nine orders of magnitude, 24 feature pairs correlate above 0.85, and the protocol feature has 133 levels. Finally, 94,928 records, or 36.8 % of the file, are exact duplicates once the identifier is disregarded. After the removal of the duplicates, 162,745 records remain, of which normal traffic accounts for 52.7 % and the worms category for 0.1 %.

The present study established a ninth property of the data, which the preceding study did not examine. After deduplication, 11,506 records, or 7.1 % of the dataset, share identical values on all 42 descriptive features with at least one record of a different category. No classifier that receives only the 42 features distinguishes such records. The conflicts are concentrated in specific categories: 81.4 % of the backdoor records, 77.9 % of the analysis records and 34.1 % of the denial of service records belong to a group of identical feature vectors that spans several categories, against 0.5 % of the normal records and 0.7 % of the shellcode records. The largest such

groups mix the reconnaissance, backdoor, denial of service, exploits, analysis and fuzzers categories. The property bounds the performance attainable on the affected categories, because a group of identical feature vectors receives a single prediction.

B. Corrections shared by all models

Four corrections were applied identically to the input of every model. The record identifier was removed, because the identifier carries no generalisable information and the ordering of the file associates identifier ranges with categories. Missing service values were encoded as an additional level, because the absence of a notable application service is itself an observation [20]. Values of the FTP login indicator greater than one were clipped to one, because any such value records the occurrence of a login. Duplicate records were removed, because a duplicate placed in both a training partition and a test partition inflates the estimated performance.

None of the four corrections learns from the data, so the corrections were applied once, before the partitioning of the data. Every remaining transformation learns a quantity from the data, such as a median, a level vocabulary or a correlation. Every such transformation was fitted on the training partition of each fold and then applied, unchanged, to the corresponding test partition.

C. Pre-processing candidates for nearest neighbour classification

The KNN input requires three decisions, and the preceding study left two of the decisions untested. The first decision is the scaling method. Four candidates were evaluated: standardisation to zero mean and unit variance, scaling by the median and the interquartile range, scaling to the range $[0, 1]$, and the absence of scaling. Interquartile scaling was expected to perform best, because the median and the interquartile range are insensitive to the extreme values present in the data.

The second decision is the encoding of nominal features. Every level was encoded as a binary indicator feature, because an integer code imposes an artificial order that Equation (1) would treat as a meaningful distance. The 133 protocol levels then produce 133 sparse indicators. Two candidates were evaluated: retention of every level, and the retention of the ten most frequent protocol levels and the five most frequent state levels with the remaining levels merged into a single level. The merge was expected to help modestly, through the mechanism described by Aggarwal *et al.* [1].

The third decision concerns two optional transformations, namely a logarithmic transformation of non-negative features with a skewness above one, and the removal of one feature from every pair with an absolute correlation above 0.95. Each transformation was evaluated as an addition to the best configuration of the first two decisions. The preceding study found that both transformations degraded performance, but the finding rested on transformations fitted to the full dataset.

D. Pre-processing candidates for tree-based models

Scaling, logarithmic transformation and the removal of correlated features were not applied to the input of the classification tree or the tree ensembles. A threshold test depends only on the order of values within a feature, so a monotone transformation leaves every candidate split unchanged. A redundant feature yields no additional impurity reduction, so the tree removes redundancy implicitly.

The encoding of nominal features does affect tree-based models, and two encodings were evaluated for each tree-based model. Integer encoding assigns one code per level, which keeps the number of features at 42 but forces the tree to isolate a level through several threshold tests. Indicator encoding allows a single test to isolate one level, at the cost of an increase to 193 features. For the classification tree, both encodings were additionally evaluated with and without the merge of rare levels.

IV. Implementation

The section describes the software, the implementation of each model, and the measures taken for reproducibility and computational tractability.

A. Software

The study was implemented in Python, version 3.13. Models, resampling, scaling and performance measures were provided by scikit-learn, version 1.9 [22], through the classes `DecisionTreeClassifier`, `RandomForestClassifier`, `NearestNeighbors`, `StratifiedKFold`, `RobustScaler`, `StandardScaler` and `MinMaxScaler`. Data manipulation was performed with pandas, version 3.0, and numeric computation with NumPy, version 2.5. The source code is publicly available.[1]

The level merge and both encodings were implemented as transformer classes with separate fitting and application methods, so that the level vocabulary of each fold was learned from the training partition only.

B. Nearest neighbour search

The KNN model was implemented through exhaustive search rather than through a spatial index. A spatial index, such as a k-d tree, partitions the feature space to avoid the computation of every distance, but degenerates to exhaustive search in high-dimensional spaces, and the indicator encoding produces between 62 and 193 features. A tuning sweep over several values of k and both weighting schemes requires only one search per fold, because the neighbours returned by an exhaustive search are ordered by distance. The list of the k nearest neighbours is therefore a prefix of the list for any larger value of k. Algorithm 1 states the procedure.

Where a neighbour lay at distance zero, every neighbour at distance zero received a weight of one and every other neighbour a weight of zero, which matches the convention of scikit-learn.

[1] https://github.com/J-Burden/26905167-rw741-assignment3

Algorithm 1 Neighbour reuse for a control parameter sweep

for each fold **do**
    Fit the pre-processing on the training partition
    Apply the pre-processing to both partitions
    Retrieve the largest candidate number of nearest training instances of every test instance, in order of increasing distance
    for each candidate value of k **do**
        Restrict every retrieved list to the first k entries
        Predict the class of greatest vote count under uniform weighting
        Predict the class of greatest total reciprocal distance under distance weighting
    end for
end for

C. Tree-based models

The classification tree was implemented through `DecisionTreeClassifier`, which grows binary trees and supports both impurity measures of Equation (2), class weighting, and the cost-complexity pruning of Equation (3). The implementation requires numeric input without missing values, which the encodings of Section III-D provide. The bagged ensemble and the RF were implemented through `RandomForestClassifier`, with q set to p for bagging. The ensemble predicts the class of greatest mean class probability over the trees, rather than the class of greatest vote count.

The examination of performance as a function of τ did not require a separate forest for every value of τ. Each forest member is trained independently of the other members, so the first t members of a forest of τ trees form a valid forest of t trees. A single forest of 300 trees was therefore trained per fold. The class probabilities of the members were accumulated in order, and the prediction of the forest was recorded after the addition of every member.

The diversity of every forest was measured on the test partition of each fold. The strength of the members was measured as the mean $\bar{F}$ of the individual trees. The correlation between members was measured as the mean pairwise agreement, defined as the proportion of instances for which two trees predict the same class [18], over every pair among 25 randomly selected trees.

D. Reproducibility and computational budget

Fixed seeds were supplied to the partitioning, to the folds, and to every tree and forest, so that every model was evaluated on identical partitions. The experiments were constrained to two processor cores, which limited the size of the tuning grids and motivated the use of 100 trees during the tuning of the RF.

V. Empirical process

The section describes the experimental procedure in sufficient detail for the reproduction of the reported results. The section first argues the choice of ensemble method, and then states the partitioning of the data, the resampling protocol, the tuning design of every model and the justification of every candidate value, the protocol of the final evaluation, and the statistical procedure used to compare the models.

A. Choice of ensemble method

The comparison was designed to reflect the effect of ensembling rather than the effect of a different hypothesis space. The RF uses the classification tree of the preceding study as the base learner, so a difference between the two models measures what aggregation and randomisation add to the same learner. A dissimilar ensemble would confound the two effects. The RF is also a strong default for tabular data: Fernández-Delgado *et al.* [10] evaluated 179 classifiers on 121 datasets and found the RF family to be the most accurate overall.

Three properties of the data informed the choice among tree ensembles. The first property is the label conflict described in Section III-A. From the perspective of any classifier, records with identical features and different categories are label noise. Boosting increases the weight of every misclassified instance, and would therefore concentrate on records that no model separates. Dietterich [8] showed that AdaBoost degraded markedly under class label noise, whereas bagging and randomised trees did not. The second property is the instability of the single tree under perturbation of the training data, which is the condition under which averaging over bootstrap samples succeeds [2]. The third property is the strong correlation between features reported in Section III-A. Correlated features cause bagged trees to select similar splits, and the random feature subset of the RF offers a direct control over the resulting correlation between trees, as expressed by Equation (4). Stacking was not considered, because stacking requires heterogeneous members, and the KNN model was both the weakest and the most expensive candidate member.

B. Partitioning and resampling

The deduplicated dataset was partitioned once, by stratified random sampling, into a development partition of 130,196 records (80 %) and a generalisation set of 32,549 records (20 %). Every tuning decision was based on the development partition alone.

Within the development partition, every configuration was evaluated by stratified five-fold cross-validation. Stratification preserves the class proportions within every fold, which is necessary for a dataset in which an unstratified partition can omit a rare class [17]. Five folds were preferred to ten folds for two reasons. The worms category comprises 137 development records, which yields approximately 27 records per test fold at five folds but only 14 at ten folds, so that the per-fold recall of the category would rest on few instances. Every model was evaluated on the same five folds, so that every comparison during tuning was paired.

The primary performance measure was $\bar{F}$ of Equation (6). The normal and generic categories jointly account for 57.3 % of the records, so a classifier that serves only the largest categories attains a high value of η while failing at the purpose

of an IDS. The measures $\bar{r}$ and η were recorded in support. The configuration of greatest mean $\bar{F}$ over the five folds was selected at every tuning stage.

C. Tuning of the nearest neighbour model

The seven pre-processing variants of Section III-C were each combined with nine values of k, drawn from $\{1,3,5,7,9,11,15,21,31\}$, and both weighting schemes, for a total of 126 configurations. Odd values of k avoid tied votes between two classes. The values are dense around five, the optimum of the preceding study, and are spaced approximately geometrically above eleven, because the decision boundary changes progressively less per additional neighbour as k grows.

D. Tuning of the classification tree

The tree was tuned in three stages. The first stage evaluated 72 configurations, formed by the combination of both impurity measures, six depth limits drawn from $\{8,12,16,20,24,\infty\}$, three minimum leaf sizes drawn from $\{1,5,20\}$, and two weighting schemes, namely uniform weights and weights in inverse proportion to class frequency. The depth limits bracket the optimum of 16 found in the preceding study at a regular step of four, and the unlimited depth serves as the unpruned reference. The leaf sizes are spaced approximately geometrically, because the effect of a leaf size limit scales with the logarithm of the limit. The second stage applied eight values of α in Equation (3), drawn from $\{0,1,2,5,10,20,50,100\} \times 10^{-5}$, to the best configuration of the first stage. The values follow a one, two, five progression that brackets the optimum of 10^{-4} of the preceding study. The third stage evaluated the four encodings of Section III-D under the best configuration of the second stage.

E. Tuning of the random forest

The forest was tuned in three stages, each of which addresses specific research questions of Section I. A joint search over every control parameter was computationally infeasible on the available hardware, so the stages form a coordinate-wise search.

The first stage addressed the third and the fifth research questions. Eight values of q, drawn from $\{1,2,4,6,10,16,24,42\}$, were combined with three weighting schemes: uniform weights, weights computed once from the training partition, and weights recomputed for every bootstrap sample. The values of q span the full range from one feature per split, which randomises the choice of split feature completely, to all 42 features, which reduces the RF to bagging. The spacing is approximately geometric, and the value of six corresponds to $\sqrt{p}$, the default recommended by Breiman [3] for classification. Trees were grown without depth or leaf size limits, following the convention of Section II-E. Every configuration used 100 trees, and the strength and the pairwise agreement of the members were recorded.

The second stage addressed the fourth research question. Under the best configuration of the first stage, six minimum leaf sizes drawn from $\{1,2,5,10,20,50\}$ and six depth limits drawn from $\{8,12,16,20,24,32\}$ were evaluated one at a time. The depth limits include the optimum of the single tree, so that the effect of the limit that the single tree required was observed directly. The mean $\bar{F}$ on the training partitions was recorded alongside the test partitions, so that the gap between the two quantified overfitting.

The third stage addressed the second research question. For the best configuration of the second stage, and for the bagged equivalent with q equal to p, $\bar{F}$ was recorded for every value of τ from one to 300. The number of trees for the final evaluation was fixed beforehand as the smallest value in $\{25,50,100,150,200,250,300\}$ whose mean $\bar{F}$ lies within 0.001 of the value at 300 trees. A final check compared the integer and indicator encodings under the selected configuration, with q held at the same fraction of the available features.

F. Final evaluation

Four models were evaluated on the generalisation set: the KNN model, the classification tree, the bagged ensemble and the RF. Every model was fitted to the full development partition under the selected configuration. The bagged ensemble shares every control parameter of the RF except q, so the comparison of the two ensembles isolates the effect of random feature selection, and the comparison of the bagged ensemble with the tree isolates the effect of bootstrap aggregation.

The tree-based models depend on a random seed, through bootstrap sampling, through the random feature subsets, and through the resolution of ties between splits of equal impurity reduction. Each tree-based model was therefore fitted under ten seeds, and the mean and the standard deviation over the seeds are reported. The KNN model is deterministic and was evaluated once. Training time, prediction time, per-class measures and the confusion matrix were recorded for every model.

G. Statistical comparison

Five pairwise comparisons were planned: the RF against each of the other three models, and the bagged ensemble against the tree. Each comparison was performed on the predictions of the models fitted under the first seed. The difference in $\bar{F}$ was assessed with 2,000 paired bootstrap resamples of the generalisation set, and the difference in accuracy with the McNemar test of Equation (7). The p-values of each test family were adjusted by the procedure of Holm [16], and a significance level of 0.05 was applied.

H. Irreducible error

The records that share identical feature values with a record of a different category cannot all be classified correctly by any deterministic classifier. An empirical ceiling was computed by grouping the deduplicated records by their feature values and assigning to every group the most frequent category of the group. The ceiling is optimistic, because the assignment uses the labels that the assignment then scores, but the ceiling identifies the categories for which the features are insufficient.

TABLE I
CROSS-VALIDATED PERFORMANCE OF THE PRE-PROCESSING CANDIDATES

Model	Candidate	Features	$\bar{F}$
Nearest neighbours	Interquartile, merged levels	69	$\mathbf{0.467 \pm 0.005}$
	Standardisation, merged levels	69	0.417 ± 0.012
	Range $[0,1]$, merged levels	69	0.414 ± 0.008
	No scaling, merged levels	69	0.293 ± 0.008
	Interquartile, all levels	193	0.463 ± 0.004
	Best, with logarithm	69	0.458 ± 0.004
	Best, with correlation filter	62	0.460 ± 0.006
Tree	Integer codes, all levels	42	0.582 ± 0.002
	Integer codes, merged levels	42	0.580 ± 0.007
	Indicators, all levels	193	$\mathbf{0.589 \pm 0.010}$
	Indicators, merged levels	69	0.582 ± 0.014
Random forest	Integer codes, all levels	42	$\mathbf{0.598 \pm 0.005}$
	Indicators, all levels	193	0.595 ± 0.006

VI. RESULTS AND DISCUSSION

The section presents and discusses the results obtained under the procedure of Section V. Pre-processing results are presented first, because every later result rests on the selected representations. The tuning results of the single models and of the RF follow, ordered by research question. The section then reports the performance of the four models on the generalisation set, examines performance per category, quantifies the effect of the label conflicts, and closes with feature importance and computational cost.

A. Pre-processing

Table I reports the best configuration obtained under every pre-processing candidate.

Scaling dominated every other pre-processing decision for the KNN model. The absence of scaling reduced $\bar{F}$ from 0.467 to 0.293, a relative loss of 37 %, because the widest-ranging features determined every neighbourhood. Interquartile scaling outperformed standardisation by 0.050, which confirms the expectation that extreme values inflate the standard deviation and thereby compress the bulk of every feature. The merge of rare protocol levels improved $\bar{F}$ by 0.004, a small but consistent gain that matches the expected effect of 124 fewer sparse dimensions. The logarithmic transformation and the correlation filter each reduced $\bar{F}$, by 0.009 and 0.007 respectively. The preceding study reported the same direction of effect, so the earlier conclusion survives the correction of the leakage. Interquartile scaling already limits the influence of extreme values, so the additional logarithm compresses useful spread. Features correlated above 0.95 are not identical, and the removed residual variation evidently carried class information.

Indicator encoding improved the classification tree by 0.008 but reduced the performance of the RF by 0.002. Under indicator encoding a single test isolates one protocol level, which suits a tree that must locate small categories within a depth limit of 16. The RF grows deep trees, which isolate a level through several threshold tests on an integer code without difficulty. The random subset of q features also samples 74 of 193 indicator features, most of which are nearly constant, so a smaller share of every subset carries information. The merge of rare levels did not help either tree-based model, which agrees with the expectation that a tree ignores a level that carries no information.

B. Tuning of the single models

The KNN model performed best with k equal to five and distance weighting, at $\bar{F} = 0.467$. Distance weighting outperformed uniform weighting at every value of k above one, and the advantage grew with k, from 0.004 at three neighbours to 0.022 at 31 neighbours. A larger neighbourhood admits more distant majority-class instances, and distance weighting reduces the influence of exactly those instances.

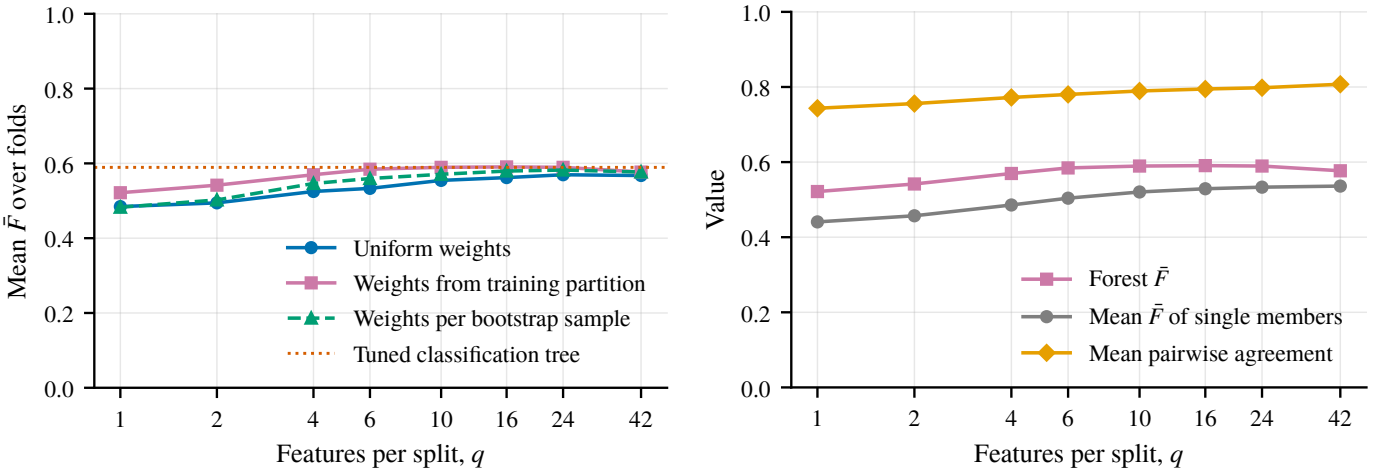
The classification tree performed best with the entropy criterion, a depth limit of 16, a minimum leaf size of one, weighting in inverse proportion to class frequency, and a complexity penalty of 2×10^{-5}. Every combination of impurity measure and weighting scheme reached the best performance at a depth of 16. The unlimited tree attained a training $\bar{F}$ of 0.849 but a test $\bar{F}$ of 0.553, against 0.715 and 0.581 at a depth of 16. The single tree therefore overfits strongly once the depth exceeds 16, which is the variance that the ensemble is expected to remove. Post-pruning added only 0.0005 to the depth-limited tree, because the depth limit had already removed most of the subtrees that the complexity penalty would remove. The configuration agrees with the preceding study.

C. Random feature subset and class weighting

Figure 1 presents the results of the first stage of the RF tuning, which addressed the third and the fifth research questions.

Figure 1a shows that $\bar{F}$ rose steeply from one to six features per split and reached the optimum at 16 or 24 features under every weighting scheme. The best configuration used 16 features, with weights computed once from the training partition, at $\bar{F} = 0.591$. The bagged equivalent, with all 42 features, attained 0.577, so random feature selection added 0.014 to plain bagging at equal weighting. The recommended default of $\sqrt{p}$, equivalent to six features, attained 0.585 and was therefore suboptimal for the dataset. The result supports the recommendation by Probst *et al.* [24] that q be tuned rather than fixed.

Figure 1b explains the location of the optimum through Equation (4). The mean $\bar{F}$ of a single member, a proxy for the strength s, increased monotonically with q, from 0.441 at one feature to 0.536 at 42 features. The mean pairwise agreement, a proxy for the correlation $\bar{\rho}$, also increased monotonically, from 0.743 to 0.807. The gain of the forest over the mean member shrank as the agreement grew: 0.081 at one feature, 0.062 at 16 features, and 0.041 at 42 features. Small subsets therefore produced diverse but weak members, large subsets produced strong but redundant members, and the optimum lay where the increase in strength no longer compensated for the loss of diversity. The agreement remained above 0.74 even at one feature per split, which is high for an ensemble of fully



(a) Forest performance

(b) Strength and agreement under class weights

Fig. 1. Effect of the number of features sampled per split

grown trees. Part of the agreement arose from records that every member necessarily misclassifies in the same way.

Class weighting remained beneficial inside the ensemble, which answers the fifth research question. At 16 features per split, weights computed from the training partition raised $\bar{F}$ from 0.562 to 0.591, a gain of 0.029, against a gain of 0.018 for the single tree under the entropy criterion. Averaging does not correct the bias of every member toward the majority categories, because the members share the same bias, so the correction must be applied to every member. Weights recomputed for every bootstrap sample performed worse than weights computed once, by 0.011 at 16 features. The experiments did not isolate the cause of the difference.

D. Tree complexity within the forest

Figure 2 presents the results of the second stage, which addressed the fourth research question.

The forest was far less sensitive to the depth limit than the single tree. A depth limit of 16 raised the $\bar{F}$ of the single tree by 0.028 over the unlimited tree, but raised the $\bar{F}$ of the forest by only 0.004, from 0.591 to 0.594. Figure 2b shows that the training $\bar{F}$ of the forest grew steadily with depth while the test $\bar{F}$ remained flat above a depth of 16. The average absorbed the additional variance of the deeper members, as predicted in Section II-C. A depth limit of eight or 12 reduced performance sharply, because shallow members lack the capacity to isolate the small categories, and averaging does not remove bias.

Figure 2a shows that a minimum leaf size of two produced the best forest overall, at $\bar{F} = 0.598$, while larger leaves reduced performance steadily. A leaf of two instances prevents a member from fitting a single conflicting record, and the effect is consistent with the report by Probst *et al.* [24] that a small leaf size limit helps on noisy data. The answer to the fourth research question is therefore that the limits required by the single tree are largely unnecessary inside the forest, and that only the mildest limit helped.

The training $\bar{F}$ of the fully grown forest was only 0.812, far below the value of one that a perfect fit of the training data would produce. The shortfall reflects the label conflicts of Section III-A: a group of identical feature vectors receives a single prediction, so every record of the group with a different category is a training error that no depth removes.

E. Number of trees

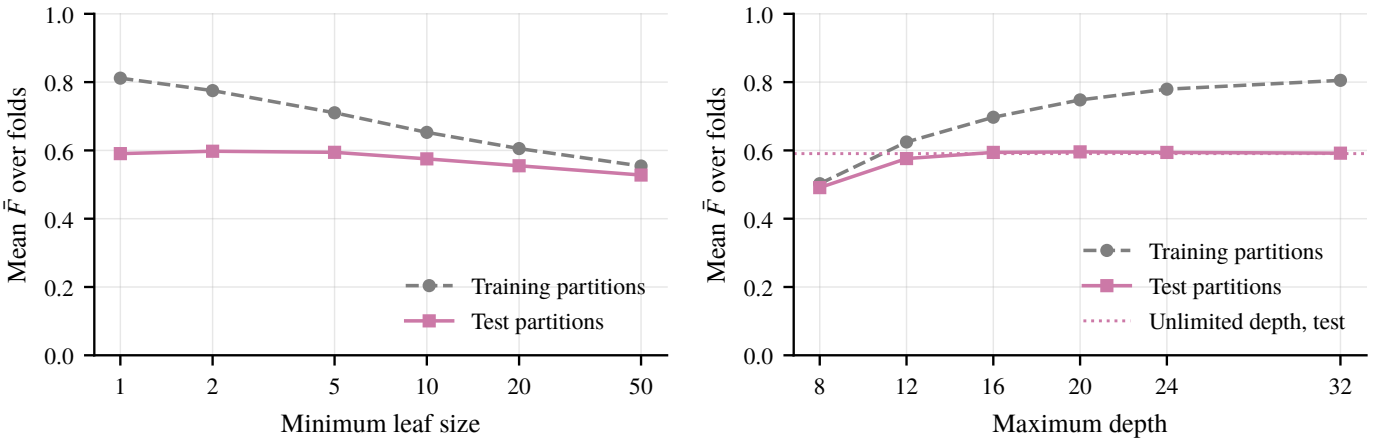
Figure 3 presents $\bar{F}$ as a function of τ for the selected forest and for the bagged equivalent, which addresses the second research question.

The forest rose from $\bar{F} = 0.520$ with one tree to 0.588 with ten trees and 0.596 with 50 trees, and then remained within 0.001 of 0.597 up to 300 trees. The selection rule of Section V-E fixed τ at 100. The highest value, 0.598, occurred at 110 trees, and the curve declined by 0.001 between 110 and 300 trees. The decline lay well within the fold-to-fold standard deviation of 0.005, so the forest did not overfit as trees were added, which agrees with the convergence result of Breiman [3]. The small non-monotone variation matches the observation of Probst and Boulesteix [23] that measures other than the error rate need not improve monotonically with τ. Most of the benefit of the ensemble was realised with 25 trees, which recovered 96 % of the gain between one and 300 trees.

The bagged ensemble converged to $\bar{F} = 0.582$. The value equals the cross-validated performance of the tuned tree on the same integer-coded input, and lies below the value of 0.589 attained by the tree with indicator encoding, which Figure 3 shows as the reference. Bootstrap aggregation of trees with all 42 features therefore removed the need for a depth limit, but did not improve on a well-tuned single tree. The entire advantage of the forest arose from the random feature subsets.

F. Performance on the generalisation set

Table II reports the performance of the four tuned models on the generalisation set, and Table III reports the paired statistical



(a) Minimum leaf size

(b) Maximum depth

Fig. 2. Effect of tree complexity limits on the random forest

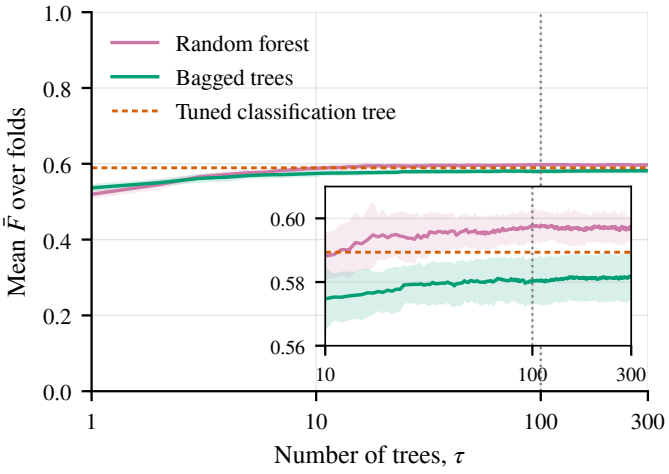


Fig. 3. Performance as a function of the number of trees

TABLE II
PERFORMANCE ON THE GENERALISATION SET OVER TEN SEEDS

Measure	Neighbours	Tree	Bagging	Forest
$\bar{F}$	0.459	0.591	0.589	**0.607**
Deviation over seeds	0	0.002	0.001	0.002
$\bar{r}$	0.453	**0.674**	0.645	0.656
η	0.769	0.752	0.781	**0.785**
Training $\bar{F}$	–	0.702	0.758	0.765
Training (s)	0.5	4.0	55.6	23.7
Prediction (s)	10.6	0.01	0.27	0.29

comparisons.

The forest attained the highest $\bar{F}$, at 0.607, against 0.591 for the tree, 0.589 for bagging and 0.459 for the KNN model. The difference of 0.016 between the forest and the tree was significant after the Holm adjustment, and the bootstrap interval of $[0.006, 0.027]$ excludes zero. The answer to the first research question is therefore positive: the RF improved on the tuned tree, by a margin that is statistically reliable but modest in size. The standard deviation over seeds was at most

TABLE III
PAIRED COMPARISONS ON THE GENERALISATION SET, WHERE u AND v COUNT THE INSTANCES CORRECT UNDER THE FIRST AND THE SECOND MODEL ONLY

	Bootstrap		McNemar test		
Comparison	**Difference in $\bar{F}$**	p	u	v	p
Forest, tree	0.016 [0.006, 0.027]	< 0.01	2072	1007	< 0.001
Forest, bagging	0.019 [0.013, 0.026]	< 0.01	705	557	< 0.001
Forest, neighbours	0.148 [0.128, 0.165]	< 0.01	2898	2381	< 0.001
Tree, neighbours	0.131 [0.111, 0.149]	< 0.01	3017	3565	< 0.001
Bagging, tree	−0.003 [−0.014, 0.008]	0.57	2027	1110	< 0.001

0.002 for every tree-based model, so the ranking does not depend on the seed. The performance of the tree and the KNN model on the generalisation set lay within 0.01 of the cross-validated estimates of the preceding study, which indicates that the leakage corrected in the present study had only a small effect on the earlier estimates.

Bagging did not differ significantly from the tree in $\bar{F}$, yet the bagged ensemble classified 2027 instances correctly that the tree missed, against 1110 in the opposite direction, and raised η from 0.752 to 0.781. The gains of bagging were therefore concentrated in the large categories, where the gains add to η but contribute little to an unweighted mean over ten categories. The forest retained the accuracy gain of bagging and added a gain in $\bar{F}$.

The three measures did not agree on a single winner. The forest led on $\bar{F}$ and η, but the tree led on $\bar{r}$, at 0.674 against 0.656. The KNN model was more accurate than the tree, with 3565 against 3017 instances classified correctly by one model only, yet trailed the tree by 0.131 in $\bar{F}$. Both disagreements arose from two specific categories.

G. Performance per category

Figure 4 presents F_j per category for the four models, and Figure 5 presents the confusion matrix of the forest.

Figure 4 shows that the forest outperformed the tree on eight of the ten categories, and outperformed bagging on nine.

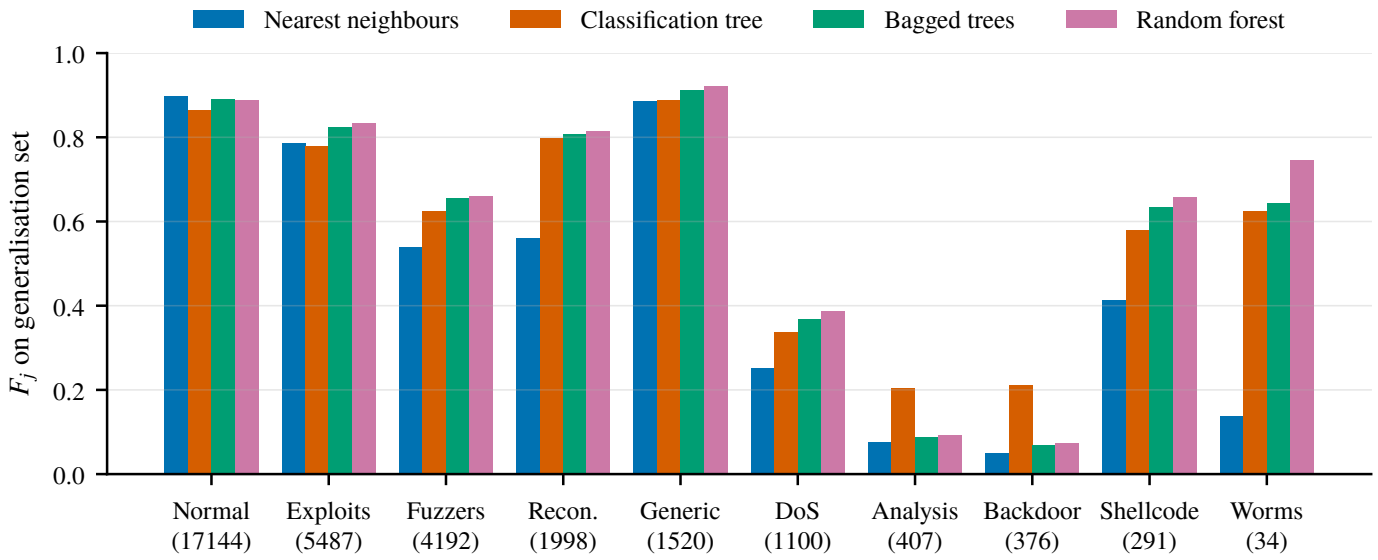


Fig. 4. Performance per category on the generalisation set, ordered by category size

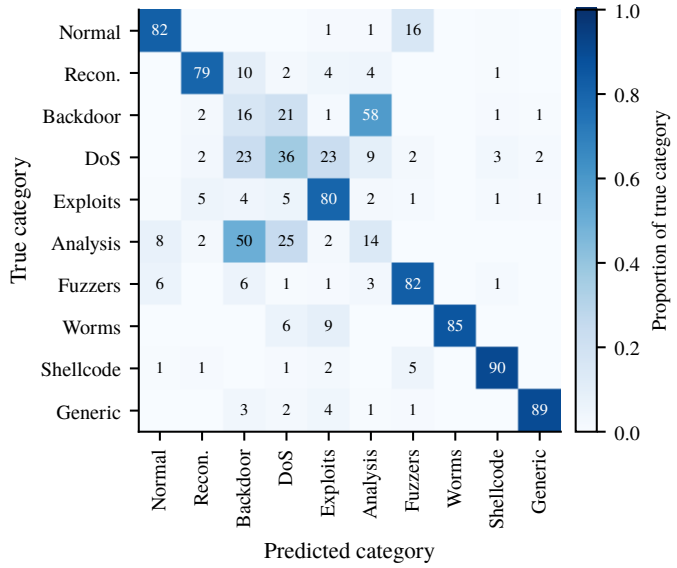


Fig. 5. Row-normalised confusion matrix of the random forest, in percent

The gains over the tree were largest on the small categories that remain separable: 0.122 on worms, 0.079 on shellcode and 0.049 on denial of service. The worms estimate rests on only 34 test records and is the least reliable of the ten. The forest performed worse than the tree on exactly two categories, backdoor and analysis, where F_j fell from 0.211 to 0.073 and from 0.203 to 0.091 respectively.

Figure 5 shows the structure of the loss. The forest assigned 58 % of the backdoor records to the analysis category, and 50 % of the analysis records to the backdoor category, while recalling only 16 % and 14 % of the two categories correctly. The two categories were therefore not merely confused but systematically exchanged. The tree, by contrast, recalled 44 % of the backdoor records and 43 % of the analysis records, at a precision of only 0.14 and 0.13. The high recall of the tree on the two categories explains the lead of the tree on $\bar{r}$, which ignores precision. The same mechanism penalised the KNN model, which recalled 6 % of the backdoor records and 5 % of the analysis records.

The remaining confusion of the forest lay in the block formed by the denial of service, exploits, analysis and backdoor categories, and between the fuzzers and normal categories, an overlap that Moustafa and Slay [21] reported in the original dataset.

H. *Effect of the label conflicts*

The systematic exchange of the backdoor and analysis categories prompted an analysis that was not planned before the evaluation. Deduplication retained at most one record for every combination of feature values and category. When a record of a conflicting group fell into the generalisation set, the development partition therefore contained the same feature values under other categories only. Of the 32,549 generalisation records, 2294, or 7.0 %, had feature values that occur in the development partition, and not one of the 2294 records had the own category among the development records with identical features. The 2294 records include 304 of the 376 backdoor records and 314 of the 407 analysis records.

A model that reproduces the development label of an exact feature match must misclassify every such record. The KNN model with distance weighting assigns all weight to an exact match, and classified 1.0 % of the 2294 records correctly. The forest and bagging, with leaves of two instances, classified 3.4 % and 3.2 % correctly. The depth-limited tree pools neighbouring groups in large leaves, and classified 13.5 % correctly, because the weighted majority of a pooled leaf occasionally matched the held-out category. On the remaining 30,255 records, $\bar{F}$ was 0.734 for the forest, 0.701 for bagging, 0.669 for the tree and 0.559 for the KNN model. The advantage of

the forest over the tree therefore grew from 0.016 to 0.065 once the conflicting records were excluded.

The analysis has two consequences. First, the evaluation protocol penalises low-bias models on conflicting records by construction, so the headline comparison understates the advantage of the forest on records that the features can separate. The lead of the tree on the backdoor and analysis categories is an artefact of the interaction between deduplication and the label conflicts, not evidence of a better model for the two categories. Second, the conflicting records explain part of the high agreement between forest members reported in Section VI-C, because every member errs identically on such records. An empirical ceiling, computed as described in Section V-H, yielded $\bar{F} = 0.808$ and $\eta = 0.944$, and limited F_j to 0.35 for the backdoor category and 0.36 for the analysis category under the accuracy-optimal assignment. The two categories are not separable from the 42 features, and no choice of classifier changes that limit.

I. Feature importance and computational cost

Permutation importance on 8000 generalisation records ranked the source-to-destination time to live first, with a reduction of 0.227 in $\bar{F}$, followed by the source bytes at 0.203, the service at 0.154 and the mean source packet size at 0.096. The impurity-based importance of the same forest ranked the time to live only fourth. The time to live takes 13 distinct values, whereas the source bytes take 9382 distinct values, so the difference between the two rankings reproduces the bias toward features with many split points reported by Strobl *et al.* [27], and supports the use of permutation importance for the interpretation of the forest.

Table II reports the computational cost of every model. The forest trained in 23.7 seconds, about six times longer than the tree, and classified the 32,549 records in 0.29 seconds. Bagging trained in 55.6 seconds, because every split evaluated all 42 features rather than 16. The forest was therefore both cheaper and better than bagging. The KNN model required 36 times longer than the forest for prediction. An IDS classifies traffic continuously and retrains infrequently, so the cost profile of the forest suits the application.

VII. Conclusions

The study evaluated a random forest (RF) for the classification of network traffic records into ten categories, and compared the RF with a bagged ensemble, a classification tree and a k-nearest neighbours (KNN) model on a deduplicated version of the UNSW-NB15 dataset. All pre-processing was fitted within the cross-validation folds, all tuning used a development partition, and every model was evaluated once on a held-out generalisation set.

The RF attained the highest macro-averaged F_1 measure on the generalisation set, at 0.607 against 0.591 for the tuned tree, and the difference was statistically significant. The improvement was modest. The tuning of the RF answered the four remaining research questions. The random feature subset was the source of the entire gain, because bagging alone matched the tuned tree. The optimum of 16 features per split balanced the strength of individual trees against the agreement between trees, and the default of six features was suboptimal. Performance converged at approximately 100 trees, and additional trees did not cause overfitting. The depth limit that the single tree required became unnecessary inside the ensemble, and only a minimum leaf size of two helped. Class weighting remained necessary, because averaging does not correct a bias that every member shares.

The most important finding concerned the data rather than the models. Of the deduplicated records, 7.1 % share identical features with a record of a different category, including approximately 80 % of the backdoor and analysis records. Combined with deduplication, the conflicts guarantee the misclassification of the affected held-out records by any model that memorises exact matches. Excluding the affected records widened the advantage of the RF over the tree from 0.016 to 0.065. The backdoor and analysis categories are not separable from the available features.

Three directions for future work follow. The first direction is the evaluation of the models on data with the original record multiplicities, grouped by feature values so that identical records never span the training and test partitions. The second direction is the merger of the backdoor and analysis categories, or the acquisition of features that separate the two. The third direction is a joint search over the control parameters of the RF, which the computational budget of the study did not permit.

References

[1] C. C. Aggarwal, A. Hinneburg, and D. A. Keim, "On the surprising behavior of distance metrics in high dimensional space," in *Proceedings of the International Conference on Database Theory*, 2001, pp. 420–434.

[2] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.

[3] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[4] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont: Wadsworth, 1984.

[5] T. M. Cover and P. E. Hart, "Nearest neighbor pattern classification," *Institute of Electrical and Electronics Engineers Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[6] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.

[7] T. G. Dietterich, "Ensemble methods in machine learning," in *Proceedings of the International Workshop on Multiple Classifier Systems*, 2000, pp. 1–15.

[8] T. G. Dietterich, "An experimental comparison of three methods for constructing ensembles of decision trees: bagging, boosting, and randomization," *Machine Learning*, vol. 40, no. 2, pp. 139–157, 2000.

[9] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman and Hall, 1993.

[10] M. Fernández-Delgado, E. Cernadas, S. Barro, and D. Amorim, "Do we need hundreds of classifiers to solve real world classification problems?" *Journal of Machine Learning Research*, vol. 15, pp. 3133–3181, 2014.

[11] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[12] J. H. Friedman, "Greedy function approximation: a gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[13] S. Geman, E. Bienenstock, and R. Doursat, "Neural networks and the bias/variance dilemma," *Neural Computation*, vol. 4, no. 1, pp. 1–58, 1992.

[14] H. He and E. A. Garcia, "Learning from imbalanced data," *Institute of Electrical and Electronics Engineers Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[15] T. K. Ho, "The random subspace method for constructing decision forests," *Institute of Electrical and Electronics Engineers Transactions on Pattern Analysis and Machine Intelligence*, vol. 20, no. 8, pp. 832–844, 1998.

[16] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[17] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proceedings of the International Joint Conference on Artificial Intelligence*, 1995, pp. 1137–1143.

[18] L. I. Kuncheva and C. J. Whitaker, "Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy," *Machine Learning*, vol. 51, no. 2, pp. 181–207, 2003.

[19] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[20] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems," in *Proceedings of the Military Communications and Information Systems Conference*, 2015, pp. 1–6.

[21] N. Moustafa and J. Slay, "The evaluation of network anomaly detection systems: statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set," *Information Security Journal: A Global Perspective*, vol. 25, no. 1–3, pp. 18–31, 2016.

[22] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[23] P. Probst and A.-L. Boulesteix, "To tune or not to tune the number of trees in random forest," *Journal of Machine Learning Research*, vol. 18, no. 181, pp. 1–18, 2018.

[24] P. Probst, M. N. Wright, and A.-L. Boulesteix, "Hyperparameters and tuning strategies for random forest," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 9, no. 3, p. e1301, 2019.

[25] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.

[26] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing and Management*, vol. 45, no. 4, pp. 427–437, 2009.

[27] C. Strobl, A.-L. Boulesteix, A. Zeileis, and T. Hothorn, "Bias in random forest variable importance measures: illustrations, sources and a solution," *BMC Bioinformatics*, vol. 8, p. 25, 2007.

[28] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

Appendix A
Acronyms

FTP	file transfer protocol
IDS	intrusion detection system
KNN	k-nearest neighbours
RF	random forest

Appendix B
Symbols

a_j	precision of class j
α	complexity penalty of cost-complexity pruning
$\mathbf{C}$	confusion matrix, with element C_{jl}
c	number of classes
χ^2	McNemar test statistic
$\delta(\cdot,\cdot)$	Euclidean distance between two instances
ε	generalisation error of a random forest
η	accuracy
F_j	F_1 measure of class j
$\bar{F}$	macro-averaged F_1 measure
G	Gini index of a node
H	entropy of a node
h	classification tree
$\|h\|$	number of leaves of tree h
k	number of neighbours consulted
n	number of training instances
n_j	number of training instances of class j
p	number of features
π_j	proportion of instances of class j at a node
q	number of features sampled at each split
$R(h)$	weighted misclassification cost of tree h
r_j	recall of class j
$\bar{r}$	balanced accuracy
$\bar{\rho}$	mean correlation between the trees of a forest
s	strength of the trees of a forest
τ	number of trees in an ensemble
u	number of instances correct under the first classifier only
v	number of instances correct under the second classifier only
w_j	instance weight of class j
$\mathbf{x},\mathbf{z}$	instances represented as feature vectors
x_j	value of feature j of instance $\mathbf{x}$